Attention Her Şeyi Görüyor Ama...
Sıra Bilgisi (Positional Encoding) Nereden Geliyor?

Atıl Samancıoğlu

1 Bu Dökümanda Ne Yapıyoruz?

Buraya kadar çok güçlü bir noktaya geldik.

Artık biliyoruz ki:

> Attention sayesinde her kelime, cümledeki diğer tüm kelimelere aynı anda bakabiliyor.

Bu, RNN/LSTM dünyasında mümkün olmayan bir şeydi. Ama tam bu noktada, çok kritik ve genellikle ilk bakışta fark edilmeyen bir problem ortaya çıkar.

Bu dökümanın tamamı şu sorunun cevabı olacak:

> "Attention kelimeler arası ilişkileri görüyor ama kelimelerin **sırasını** nereden biliyor?"

Bu soruya net bir cevap vermeden Transformer mimarisi **eksik** kalır.

2 Ana Örneğe Geri Dönelim (Yine Aynı Cümle)

Her zamanki gibi aynı cümleyle devam ediyoruz:

> *"I did not say he stole the money."*

Şimdi şu iki cümleyi karşılaştıralım:

> *"I did not say he stole the money."*
> *"Not did I say he stole the money."*

Kelimeler neredeyse aynı. Ama anlam açıkça farklı.

Bu bize şunu söylüyor:

> Dil sadece "hangi kelimeler var?" sorusundan ibaret değil.
> "Hangi kelime nerede?" sorusu da en az onun kadar önemli.

3 RNN'lerde Bu Problem Neden Yoktu?

Burada kısa bir geri dönüş yapalım.

RNN, LSTM ve GRU'da sıra problemi yoktu çünkü:

- Model kelimeleri zaten sırayla işliyordu
- Zaman adımı (t) doğrudan sırayı temsil ediyordu

Yani sıra bilgisi **mimariye gömülüydü**.

Transformer'da ise durum tamamen farklı.

4 Transformer'da Kritik Kayıp

Attention mekanizması şunu yapar:

> "Bu bir küme. Herkes herkese bakabilir."

Ama bu kümede:

- İlk kelime hangisi?
- Son kelime hangisi?
- İki kelime arasındaki mesafe ne?

Bu soruların **hiçbirinin** cevabı yoktur.
Eğer sadece embedding'leri alıp attention'a verirsek, model için şu iki durum **ayırt edilemez**:

> "dog bites man" ve "man bites dog"

Bu noktada çok net bir problem var:

5 Kritik Soru

> "Sıralı işlem yapmadan, sırayı modele nasıl öğretiriz?"

Transformer mimarisinin en zarif çözümlerinden biri tam olarak burada devreye girer: **Positional Encoding**.

6 Positional Encoding'in Temel Fikri

Fikir aslında çok basit ama çok güçlüdür:

> "Her kelimenin embedding'ine, onun cümledeki pozisyonunu temsil eden bir sinyal ekleyelim."

Burada özellikle "ekleyelim" kelimesine dikkat et.

6.1 Neden Ekliyoruz?

Çünkü:

- Attention mekanizmasını değiştirmek istemiyoruz
- Sıra bilgisini embedding seviyesinde vermek istiyoruz

Yani model hâlâ saf attention yapacak, ama embedding'ler artık "konum bilgisi" taşıyacak.

7 Pozisyonu Sayıyla Vermek Yetmez

İlk akla gelen fikir şu olabilir:

> "İlk kelimeye 1, ikinciye 2, üçüncüye 3 verelim."

Ama bu çok kötü bir fikirdir.
Neden?
Çünkü:

- Model, 10 ile 20 arasındaki farkı anlamsal sanabilir
- Uzun sekanslarda genelleme bozulur

Bizim istediğimiz şey şu:

> "Model pozisyonları karşılaştırabilsin ama mutlak sayılara takılıp kalmasın."

İşte burada sinüs ve kosinüs devreye girer.

8 Sinüs ve Kosinüs ile Pozisyon Kodlamak

Transformer'ın orijinal makalesinde kullanılan positional encoding şu şekildedir:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

Bu formüller ilk bakışta korkutucu görünebilir. Ama durup sezgisel olarak bakalım.

8.1 Bu Formül Ne Yapıyor?

Bu yapı sayesinde:

- Düşük boyutlar: hızlı değişen sinyaller (yakın pozisyonlar)
- Yüksek boyutlar: yavaş değişen sinyaller (uzak pozisyonlar)

oluşur.
Yani her embedding boyutu, pozisyona farklı bir "frekansla" tepki verir.

8.2 Neden Bu Akıllıca?

Çünkü bu yapı sayesinde model şunu öğrenebilir:

> "Bu kelime, diğer kelimelere göre ne kadar uzakta?"

Üstelik bunu:

- Eğitimde görmediği uzunluklar için bile
- Genelleme yaparak

başarabilir.

9 Embedding + Positional Encoding

Artık her kelime için iki bilgi var:

- Kelimenin anlamı (embedding)
- Kelimenin konumu (positional encoding)

Bu ikisini şu şekilde birleştiririz:

$$x_t = \text{Embedding}(w_t) + PE(t)$$

Bu denklem çok önemlidir.

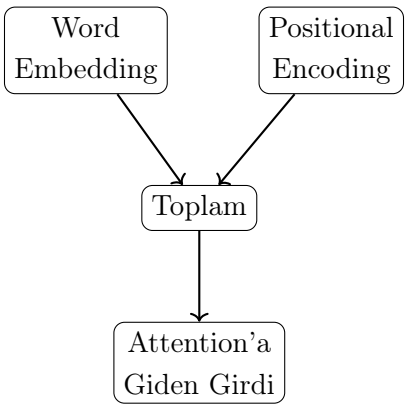
9.1 Neden Toplama?

Çünkü:

- Model, iki bilgiyi aynı anda görür
- Attention mekanizması değişmeden kalır

Yani sıra bilgisi, attention'a **dolaylı** olarak aktarılır.

10 TikZ ile Sezgisel Bir Görsel



Şekil 1: Embedding ve pozisyon bilgisinin birleştirilmesi

Bu görseli okurken şunu düşün: Attention sıralı değil, ama girdisi sıralı bilgi taşıyor.

11 Ana Örneğe Son Kez Bakalım

"not" kelimesi:

- Hem anlam olarak olumsuzluk taşıyor
- Hem de fiilden önce geldiği için kritik

Positional encoding sayesinde model şunu ayırt edebilir:

> "Bu olumsuzluk, fiilden önce mi sonra mı gelmiş?"

Bu ayrım, çeviri ve anlam için hayati.

12 Burada Durup Büyük Resmi Netleştirelim

Bu dökümanın sonunda şu fikir çok net olmalı:

> Transformer sıralı çalışmaz ama sıra bilgisini bilinçli olarak modele verir.

Attention ilişkileri yakalar, positional encoding ise bu ilişkilerin **nerede** kurulduğunu söyler.

13 Sonraki Dökümanda Ne Yapacağız?

Artık elimizde şunlar var:

- Multi-Head Attention
- Positional Encoding

Ama hâlâ eksik bir şey var.
Bir sonraki dökümanda şu soruya odaklanacağız:

> "Bu kadar karmaşık bir bloktan sonra, model neden kendisiyle toplama yapıyor? (Add & Norm)"

Yani: **Residual Connections ve Layer Normalization**.